easily match these two identifiers as likely being equal, but to a computer they are completely different. When you're adding context by matching records between different systems, you will often need an approach to match similar identifiers.

TRY IT OUT: MATCH TWO DATASETS

The easiest way to try and find similar items is to compare every item in your data set with every other item. Listing 5.1 shows an approach that uses a technique known as the *Levenshtein distance* which is a measure of the number of edits that need to be made to one string to convert it to another as a way to fuzzily match identifiers. You can use this approach to match items with identifiers that are similar, but not exactly the same.

> ***NOTE***
>
> `The fuzz.token_set_ratio` function in the example is a variation that ignores word order and duplicate words, often providing a better similarity score for identifiers.

Listing 5.1 A simple example of fuzzily matching identifiers which is a common approach to linking data records for contextualization.

```
from thefuzz import fuzz

identifiers = [ #1
    "CUST-001234", "CUSTOMER-001234", "cust_001234",
    "PROD-ABC123", "PRODUCT-ABC123", "Product ABC-123",
    "EMP-789456", "EMPLOYEE-789456", "emp_789456"
]

print("Fuzzy Matches (score >= 80):") #2
for i, id1 in enumerate(identifiers):
  for id2 in identifiers[i+1:]:
    score = fuzz.token_set_ratio(id1, id2)
    if score >= 80:
      print(f"{id1:15} ↔ {id2:15} ({score})")
```

#1 Set of sample identifiers that need to be fuzzily matched.
#2 Find fuzzy matches (identifiers that are similar and likely to refer to the same entity)